Lorca: Exhaustive Engineering and Reliability Assessment

Audit date: October 29, 2026. Prepared for Lalith. Local repository: /Users/deve/Downloads/lorca. Audited HEAD: eab2978, plus the existing uncommitted working tree. This is an assessment of the actual working tree, not a claim that every audited feature is published on main.

This report answers what we are trying to build, what exists, what works, what fails, why progress has not translated into dependable daily use, and the smallest practical way forward. It combines source inspection, saved incident records, fresh automated tests, real local-model experiments, a native application acceptance run, and browser validation of generated CSS.

No production implementation was changed during this audit. Test fixtures, diagnostic scripts, build outputs, and this report were created separately. Fingerprints of 166 Rust, Swift, and Python source/test files matched before and after the audit. No commits, pushes, Google Sheet updates, or emails were sent as part of this assessment.

Reading guide

Sections 1-4 give the conclusion, target, development history, and architecture. Sections 5-15 examine tools, authority, objectives, questions, the damaged frontend incident, context, inference, and model limitations. Sections 16-20 cover UI/lifecycle, Job Apps, architecture quality, development mistakes, and blockers. Sections 21-25 give the minimal repair sequence, acceptance criteria, full test results, limitations, and reproducible evidence references.

The central finding is that Lorca has substantial working infrastructure but an unreliable agreement between the user's request, the model's proposed work, and the harness's judgment of completion. Both the harness and generated code contribute. A model upgrade alone will not fix this; neither will adding more verification labels without meaningful checks.

1. Executive assessment

Lorca is a real local coding-agent application, not a mock interface. It can inspect an authorized workspace, discover tools, create and patch files with approval, run supported Cargo checks, retain execution evidence, resume a structured question, and present the work through a native macOS interface. Fresh tests demonstrated successful end-to-end execution, including a correct Rust repair and independent passing tests.

It is not yet a dependable general-purpose computer worker. The same audit also reproduced incorrect permission interpretation, an explicit cancellation that did not replace the old objective, unnecessary question loops, and frontend code that was labeled complete even though Safari rejects its generated CSS. These are concrete defects, not merely a subjective sense that a local model is weak.

The most revealing comparison used the same simple Rust subtraction bug. With 'Do not edit other source files,' Qwen proposed the correct fix but Lorca blocked every edit, including the requested file. With 'Keep changes limited to src/lib.rs,' the task completed and the tests passed. This isolates a harness failure from model capability. Conversely, a precise rainbow-slider prompt led the model to generate invalid CSS. That isolates a model/output-quality problem that the current checks did not catch.

The architecture should be retained. The next work should concentrate on a small number of contracts: active objective and scope, question/resumption/cancellation, bounded edits against actual content, and evidence-based completion. Those repairs do not require another agent, a memory framework, a planning hierarchy, or a Job Apps rewrite.

What is misleading today is the distance between a successful tool operation and a finished user outcome. An atomic, authorized patch can still delete essential content. A syntactically valid JavaScript program can still construct invalid CSS. A 'Completed' status can still follow no required edit if the objective recognizer misses the user's phrasing.

The proper reliability goal is not that every model output is correct. It is that ordinary errors are recoverable, user intent survives, unauthorized actions remain blocked, and the system distinguishes applied work, tested work, unverified behavior, and an actual blocker. It must avoid silently substituting a smaller task or confidently claiming success that the evidence does not establish.

2. The actual product goal

The intended experience is straightforward: choose a local model and a workspace, describe a task naturally, answer meaningful questions when necessary, review consequential changes, and let Lorca carry the work through useful checks. You should be able to continue in the same conversation, change direction, add another feature, and return to earlier decisions without manually managing inference requests or tool schemas.

The agent should be modular: a tool should have a clear implementation and registration point; inference, authorization, context, execution, and presentation should have understandable responsibilities. It should be simple: avoid a second planner, a second agent, and duplicated state. It should be powerful: possess the tools required for the work it claims to support. It should be reliable: preserve the task and evidence, recover from routine mistakes, and stop honestly when it cannot finish.

Your willingness to chunk work is an advantage. Lorca does not need to create an entire frontend in one enormous patch. It can inspect the current file, agree on a visible behavior when that matters, make one bounded change, check it, and continue with the next change. Clarification is useful when it resolves a real decision; it is not useful when the system asks you to restate a clear request after an internal timeout or malformed tool call.

A usable daily coding workflow has five observable outcomes. First, it understands the requested result and boundaries. Second, it chooses actual available capabilities. Third, it executes approved work against current files. Fourth, it performs appropriate checks or states exactly which behavior remains unverified. Fifth, it gives a concise, accurate handoff and accepts the next instruction without dragging along an obsolete task.

Universal model support should mean a common contract across supported models, with capability negotiation for optional features. It cannot mean that every arbitrary model is equally competent or that every backend exposes the same reasoning and tool features. Lorca can remain model-neutral while testing a small supported set and reporting capability limits explicitly.

The target is not 'verify every action with another LLM.' It is 'use cheap deterministic checks where available, model judgment where necessary, human input where preferences matter, and truthful uncertainty everywhere else.' This distinction keeps the implementation realistic and small.

3. What has actually been built

Git history records rapid expansion across several real subsystems. On October 23, the project established initial UI, terminal UI, and the macOS dashboard/Mini checkpoint. October 24 added a shared Rust core across CLI and GUI, approved workspace tools, unified conversations and execution, streaming, compaction, and submission/backend safeguards. October 25 added native Job Apps research, Gmail drafts and two-way Sheets synchronization, followed by job numbering and ordering changes.

October 26 added cross-Mac numbering/settings work, the native Job Apps review/outreach/resume workflow, oldest-pending-action guidance, and crisp resume rendering. October 28 recorded the modular task harness, adaptive compaction changes, and proportional UI zoom. Further tool-chest, recovery, working-checkpoint, and inference changes exist in the current dirty working tree.

This is a significant amount of software beyond 'connect one model to one chat window.' It includes data persistence, native rendering, context projection, permissions, process lifecycle, idempotency, provider transport, model capabilities, Google integration, resumes, synchronization, and two different user-facing work domains. Those features are useful, but their presence does not prove the central general-agent workflow is reliable.

What went right is the reuse of a shared Rust execution core instead of independent terminal and native agents; real tool handlers instead of simulated outputs; approval and workspace enforcement; bounded results; and independent Job Apps structure. The new code has not required a distributed service architecture or another agent framework.

What went wrong is the order in which readiness was judged. Infrastructure milestones and successful examples were repeatedly treated as product completion. Features accumulated while the central acceptance case - understand a request, edit the correct content, check the result, and handle a follow-up - still had gaps. The development history contains legitimate progress, but also repeated polish, repair, reversion, rebuild, and integration work around an unstable behavioral contract.

The repository history cannot establish exact hours spent, credit cost, or the proportion of effort attributable to each mistake. This report does not invent those figures. It does establish that the scope became much broader than the initial agent loop and that tests for that loop did not catch several ordinary user-language cases.

4. Architecture and responsibility boundaries

The native SwiftUI/AppKit interface communicates with a local Rust backend. The shared core manages conversations, model operations, task execution, and persisted state. The transport is an authenticated loopback protocol; the current protocol version is 3. Request/task identity and existing idempotency handling reduce accidental duplicate execution.

The execution path can be summarized as: user message -> conversation/task selection -> objective and scope -> context projection and tool schemas -> Ollama generation -> argument parsing and validation -> trusted tool preparation/approval -> execution -> recorded evidence -> continuation or completion. Failures can happen at every boundary, so a single 'LLM problem' label is inadequate.

Objective is responsible for what the user wants and which task is active. Context is responsible for what the model currently sees, not for changing the user's goal. Authority controls what may run and where. Verification records what actually happened and decides what evidence supports the requested outcome. Execution coordinates those parts. The model proposes actions and interprets results, but generated statements are not execution evidence.

The current design already has the necessary places to make focused repairs. The important issue is disagreement between them. A natural-language scope checker can forbid an edit that the objective requires. A question can be emitted as plain text while the loop expects a native tool. A generic failure question can be persisted under a different status from a genuine waiting question. A completion recognizer can decide there is no edit requirement even when the user plainly requested a fix.

The application should preserve these boundaries rather than moving all judgment into the model or all interpretation into Rust string matching. Mechanical checks belong in Rust. Preferences and semantic interpretation require model/user participation. Tool evidence should remain structured and authoritative. The final answer should be constrained by that evidence, with functional uncertainty stated rather than hidden.

The existing structure is therefore a suitable foundation. The problem is not that Lorca lacks enough layers. It is that a few small but important state transitions and interpretation rules are currently too brittle.

5. Tool chest: what the five drawers really provide

Files contains read_file, list_directory, search_files, apply_patch, and git_diff. Terminal contains exec_command and write_stdin, with shared session/process logic. Vision contains view_image. Web contains fetch_url and conditionally available web_search. Agent contains tool_search, update_plan, request_user_input, and request_permissions. The folders physically reflect the architecture and a central registry describes available tools.

File operations are real. Workspace-relative access, capability-based filesystem handling, symlink restrictions, patch preparation, stale-content checking, atomic writes, and readback protect important mechanical boundaries. These safeguards should remain. They prevent many classes of unauthorized or inconsistent edit; they do not prove the edited program behaves correctly.

The Terminal drawer is much narrower than its public name suggests. exec_command currently accepts Cargo verification operations rather than arbitrary terminal work. It cannot run a Node syntax check, Python server, shell script, browser-launch command, or ordinary project scaffolding command. A direct probe accepted Cargo test for approval and rejected Node and Python examples. write_stdin supports existing process interaction; it does not expand what can initially be launched.

This matters for the user's expectation of 'do my work with it.' A frontend task ordinarily needs some combination of JavaScript/CSS validation, a local server, and a browser. The present general agent cannot perform that complete sequence through its tools. git_diff is available, but it is not a general Git mutation interface. There is no general mkdir/rename/delete tool, so scaffolding arbitrary nested project structures is also constrained.

Vision inspects available local images through the model pipeline. It is not a general screenshot-capture or browser-control facility. PNG/JPEG size and dimension checks are useful, and unsupported capabilities should fail clearly. Fetching a public HTTPS document is not equivalent to opening a JavaScript-heavy authenticated website. fetch_url does not provide browser state, cookies, interactive controls, or rendered-page verification.

web_search is only useful when its external configuration exists. Job Apps having an Exa integration does not automatically make general-chat search configured. Missing integration must remain explicitly unavailable. The clean drawer structure is a success; the gap is capability coverage and expectation, not a need for more organizational layers.

6. Limits that explain otherwise confusing behavior

read_file returns bounded excerpts: at most 8,192 bytes per call, with offset and nextOffset supporting continued reading. The shared file size limit is 128 KiB. Tool arguments are bounded at 64 KiB. These limits are reasonable defenses against excessive context and runaway writes, but the agent must account for them when planning work.

A verified file path means Lorca knows that reference is allowed and discovered. It does not mean the model has read the entire file. An excerpt can end in the middle of a document, and the visible truncation label is not the same mechanism as context compaction. Compaction reduces prompt history; it does not magically fetch the remainder of a file or restore omitted source content.

The 100,000 colored-name example exposes this capability mismatch. Plain text cannot encode visible colors on its own; HTML or RTF would be appropriate if the user chooses that format. But generating hundreds of kilobytes through the present patch-only path would exceed existing bounds. The agent needs an appropriate authorized generator capability or an honest blocker. Writing ten repetitions instead and calling the task complete is not an acceptable substitute.

Likewise, a Desktop destination outside the selected workspace is a deliberate authority boundary. The right response is to explain the needed workspace or permission action. request_permissions cannot itself grant broader access. The model must not claim it can access a path merely because it appears in a user message.

These are not reasons to remove limits. They are reasons to make the available capabilities and limits visible before the model chooses a strategy. Small incremental patches fit the current architecture much better than huge old_text/new_text replacements. Large generated artifacts require a different appropriate tool, not larger and larger chat payloads.

The general principle is that an unavailable operation should be distinguished from a model mistake and from missing user information. The current system sometimes funnels all three into generic 'What can you provide to unblock it?' questions, making the user troubleshoot internal limitations without a clear explanation.

7. Confirmed harness defect: scoped authority becomes a global ban

Fresh experiment: a disposable Rust project contained an add function that subtracted. The request asked Lorca to fix that function, run cargo test, and 'Do not edit other source files.' Qwen generated the correct patch to src/lib.rs. Lorca rejected it. The source stayed unchanged and an independent cargo test still exited 101.

The relevant authority logic recognizes phrases including 'do not edit' globally. It does not preserve the qualifying word 'other' as a path-scoped restriction. Consequently, a request permitting one edit while forbidding unrelated edits becomes a request forbidding all edits. This is the opposite of the user's intended safety boundary.

A controlled positive-wording comparison asked for the same repair but said 'Keep changes limited to src/lib.rs.' The task completed in about 16 seconds, the correct change was applied, and both Lorca's Cargo run and an independent check passed. The negative-wording case took about 95 seconds and ended waiting for input after blocked edits and failed tests.

This is strong evidence against blaming the model for every failed coding task. The model's proposal was correct in the blocked case. The harness prevented useful execution. Changing temperature, reasoning effort, model size, or output allowance does not repair this policy interpretation.

The repair should preserve explicit authority and scoped restrictions, not weaken authorization. A small representation of allowed targets and forbidden effects is more defensible than a growing list of global English substrings. A model can help interpret scope, but trusted execution must check a concrete scope once established. Ambiguity should produce a targeted question, not a silent global prohibition.

This audit did not change that code. It demonstrated the defect with the public implementation and a live model. It is a high-priority blocker because careful users naturally say 'do not edit anything else.' That phrasing should make the agent safer, not make the requested edit impossible.

8. Confirmed harness defect: completion depends on phrasing

The verification system derives requirements from the user request using heuristics. That is useful for simple mechanical obligations, but several natural requests produce no edit requirement. A direct probe supplied no tool evidence at all. 'Fix index.html' correctly remained unsatisfied. 'Make the gradient control change hue across the rainbow' was considered satisfied without any work.

More seriously, the exact style of the observed follow-up - 'still looks so bad and the slider still doesn't work ... fix it' - did not establish an edit requirement. This explains how the incident could finish with a Completed status after an obviously wrong result. The harness had lost a meaningful outcome to check, even though the conversation and user intent still made it clear that code repair was requested.

This is not a reason to demand every original message verbatim forever. It is a reason to retain a concise, current statement of the requested outcomes and update it when the user steers the task. 'Fix it' can refer to the active frontend task. Treating only the latest message as an isolated keyword puzzle is inadequate for a continuing agent conversation.

Another direct probe found the opposite error. For 'Run cargo test and report the actual exit code and failures; do not edit files,' actual execution evidence with exit code 101 remained unsatisfied. The requirement effectively expects passing tests, although the user asked to run and report them, not repair them. That conflates execution/reporting success with the tested program's success.

The appropriate distinction is small: run-and-report, repair-and-verify, explanation, and waiting-for-decision are different requested outcomes. A failed test result can fully satisfy the first and block the second. Applied bytes can support an edit claim while leaving behavior unverified. A result summary should reflect those distinctions.

Verification must not be weakened to 'the model says done.' But it also must not impose a different task from the user's request. Both false positives and false negatives currently occur, which is why the user sees both premature completion and unnecessary incompletion.

9. Questions, clarification, and explicit changes of direction

The existing structured request_user_input path can work well. In a fresh test the model was explicitly instructed to use that tool before choosing a filename. Lorca entered WaitingForInput. The user answer selected CHOSEN.md with empty contents. Lorca resumed and created exactly that empty file, then completed. This is a working building block worth retaining.

Ordinary conversational clarification is less reliable. Asked simply to 'Create a file,' Qwen produced sensible questions about filename, content, and whether a file already existed. The harness's plain-text question detector is narrow: it limits length, expects a question ending, requires exactly one question mark, and checks particular phrasing. A reasonable multi-question response was not recognized.

The loop therefore treated clarification as an unfinished edit. It prompted continuation and Qwen repeated its questions. The case ended in a generic incomplete route rather than a clean waiting state. Repeating the experiment reproduced the same pattern. These experiments did not proceed to the answer stage, so they are evidence about question recognition, not proof that all resumption is broken.

A separate fresh test did exercise cancellation. While a native question was pending, the user explicitly said to cancel the previous file task, start a new request, answer 2+2, and not edit or run commands. Core automatically appended that message as a 'User decision' to the old objective. The obsolete edit requirement stayed active. The result was Incomplete instead of a clean answer to the new request.

This matters directly to your goal of working in one chat for a long time. It is not only a memory-size issue. The system must understand whether a message answers a pending question, changes the current task, cancels it, or starts a new task. A tiny explicit state transition is preferable to a new memory framework.

There is also inconsistency between native waiting questions and generic recovery guidance that invokes request_user_input yet ends as Incomplete. The user should not need to know which internal route produced a question in order to answer it. Questions should be concise, recommend a direction when appropriate, and resume consistently. Routine tool errors should be recovered internally before asking the user to solve them.

10. Production frontend incident: what actually happened

The saved conversation and current playground file provide a much more precise explanation than 'the model forgot.' The original file was 9,216 bytes. During the first repair, the model attempted a large near-whole-file replacement whose old_text did not match. It then made smaller successful changes, producing an intermediate 9,727-byte file.

During the later High-effort repair, the model replaced an 8,191-byte prefix with an 8,703-byte replacement, producing 10,239 bytes. The same proposal was encountered again, but idempotency marked it already executed and did not apply a second side effect. That was a safeguard working correctly.

The damaging final patch used the same 8,191-byte prefix as old_text and an empty replacement. That is a valid exact match. It removed the main HTML document and left a 2,048-byte fragment. The current damaged file contains trailing script/document fragments rather than a complete opening HTML document. Its 2,048 bytes correspond to 2,020 Unicode characters.

The final model answer claimed a 10,239-byte result. That number was not wholly invented: it was an intermediate successful state before the destructive deletion. The failure was selecting stale evidence and presenting it as the final state. The final tool result reflected the smaller file, but the summary did not respect it.

No recorded compaction or explicit output-truncation event occurred in those two repair tasks. The evidence does not support calling automatic compaction the root cause of this incident. Bounded file excerpts, very large replacements, a later destructive exact match, missing functional checks, and weak completion requirements explain the observed sequence more directly.

Exact matching protected against editing unexpected bytes; it did not protect against deleting the wrong amount of correctly matched content. Approval authorized an operation; it did not certify that the proposed diff preserved the requested features. The user's approval is not evidence that the model's edit was semantically correct.

The concise root cause is a compound failure: poor model patch strategy plus insufficient outcome validation plus stale final reporting. The model made the bad change, the harness allowed an approved mechanically valid change, and the completion mechanism did not catch the missing result. No single larger output limit would solve all three.

11. Fresh frontend experiments: what syntax checks miss

Two fresh runs used the same small disposable HTML fixture and requested a full-rainbow gradient slider, preservation of controls, and removal of stray raw JavaScript. Both completed, produced the same 708-byte output, preserved basic structure, removed the stray fragment, and passed JavaScript syntax checks. This is evidence of repeatable useful behavior on a small task.

However, the generated slider rotated a fixed seven-color gradient. It did not change hue based on slider position. The broad 'full rainbow' wording admits some interpretation: adding seven visible color stops is a real improvement. Under a hue-sweep acceptance criterion it is incomplete, but this should not be overstated as an unambiguous failure of a perfectly specified prompt.

A third run removed that ambiguity. It specified red at the minimum, green near one-third, blue near two-thirds, and hue changes rather than rotation. Lorca completed, and the JavaScript still passed syntax checks. The generated JavaScript constructed a malformed CSS gradient string with an extra '(rgb(' stop and mismatched parentheses.

A Node VM with DOM stubs showed the actual strings produced at four slider positions. That test alone does not parse CSS or render a page, so it was not treated as browser evidence. A separate local Safari diagnostic then called CSS.supports('background', value) on those exact strings. All four generated samples returned false; a valid control gradient returned true.

This confirms a concrete functional defect that a JavaScript parser cannot detect. The program can be legal JavaScript while its generated CSS is rejected by the browser. The dark-mode event toggled a class in the stub test, but that is not proof of readable dark-theme styling; class changes and rendered appearance are different checks.

These results illustrate the required verification philosophy. Check the behavior that matters when a practical oracle exists. For a slider, observe resulting values and browser acceptance, not just file existence. For a frontend, visual review or browser interaction is useful; a generic 'verified' patch flag is not enough. Lorca currently lacks those browser capabilities in its own general tool chest, so it should report that limit rather than imply visual testing occurred.

12. Compaction, persistent checkpoints, and long chats

Lorca already has context budgeting, history projection, compaction, task state, execution evidence, and optional working checkpoints. This is more than a raw chat wrapper. The important question is what each mechanism actually preserves.

Current compaction is primarily deterministic reduction of older content and observations. Bounded excerpts are retained rather than a guaranteed semantic summary of every important decision. Inspection found reduced tool observations, shortened narrative, bounded older JSON, and a recent-message tail. This can control prompt size, but it is not equivalent to a perfect task memory.

The full conversation remaining on disk does not mean the model can see every relevant earlier detail. Conversely, a low context percentage does not prove the right information is present. A 16% context display can coexist with an obsolete objective, a missing requirement, or an invalid generated patch. Those are semantic/state issues rather than capacity exhaustion.

Working notes through update_plan are useful for decisions, uncertainty, and next action, and they are distinct from trusted execution evidence. They are optional tool output, not an automatic guarantee that every approaching generation limit has a checkpoint. In the inspected historical collection, update_plan did not appear; many records predate the latest feature, so that is not a fair adoption-rate benchmark.

The objective currently retains the instruction text and appends user decisions. The effective task has an 8,000-byte limit. This is bounded and explicit, but it is not yet a concise, actively revised model of a long conversation with many separate assignments. Repeated decisions can grow it, and automatic question linkage can preserve a task the user explicitly canceled.

The useful minimal direction remains a small current-task checkpoint: requested outcomes, essential constraints, user decisions, next step, and unresolved blocker. Execution evidence remains separate and authoritative. Completed tasks can become concise history. New tasks can replace the active objective without deleting the chat. No inaccessible internal thinking state needs to be retained.

This audit exercised small tasks and inspected compaction regressions; it did not establish that arbitrarily long chats are reliable. The existing long native acceptance fixture has stale paths and was not accepted as a valid long-context benchmark. A truthful claim is 'mechanisms exist and several tests pass,' not 'memory loss is now impossible.'

13. Generation limits, malformed calls, and recovery

The output-budget mechanism is real and useful. It reserves prompt room, chooses a generation allowance from context capacity, supports a larger recovery allowance, and can react to explicit interrupted generation. The current policy starts around a fraction of the context window, caps normal and recovery allowances, and includes a safety reserve. It is not an unlimited-generation mechanism.

A separate provider failure previously mattered: Ollama can return HTTP 500 containing an invalid/truncated tool-call JSON error instead of a normal length-stop response. Narrow recognition and bounded recovery for that specific error exist in the current implementation and have regression coverage. Unrelated server errors should not be treated as safe-to-repeat truncated generation.

Incomplete JSON must never become an executable action. The current parsing, validation, and trusted preparation boundary are important successes. Retrying a full malformed patch blindly would create loops and unnecessary context; the recovery guidance should favor smaller incremental edits and a larger allowance only when justified.

There are still limits to what can be inferred. A server error may obscure exactly how many tokens were generated or why the model stopped. No client can guarantee a model writes a useful checkpoint just before every limit. A checkpoint itself requires valid complete output. The safe fallback is to discard incomplete actions, retain the objective and verified work, and retry within a bounded policy.

Budget management also does not fix semantic mistakes in complete output. The 8,191-byte destructive prefix patch was complete and mechanically valid. The invalid CSS was inside valid JavaScript. A larger generation budget might merely produce a larger wrong patch. Good recovery and good edit strategy are complementary, not interchangeable.

The loop already has iteration, tool-call, failure, and no-progress limits. These should remain bounded. What needs improvement is deciding what counts as useful progress: a new paragraph, a repeated clarification, or an applied but destructive edit is not necessarily progress toward the user outcome. The next step should be selected from the remaining requested work and actual observations, not from a generic instruction to keep going.

14. Model compatibility, thinking, and hardware

Lorca has a reusable inference boundary, but the current implementation is centered on Ollama transport. That is model flexibility within a provider, not a native MLX backend. A model advertising completion, tools, vision, and thinking is not automatically equally reliable at all four, and optional thinking controls must follow what the installed model/provider actually accepts.

Qwen's installed local model advertises boolean thinking controls. The fresh thinking smoke test returned valid read_file calls in Off, Nominal, and High. That demonstrates request/response compatibility; it does not prove better coding quality with High. The Gemma installation in prior inspection did not advertise the same explicit controls, and an explicit unsupported setting can be rejected. Capability negotiation is necessary even in a model-neutral agent.

Temperature is also only one factor. Lorca's adapter currently supplies a low temperature rather than simply inheriting every model-file default. Lower randomness can improve consistency, but it does not make a model obey a complex patch contract perfectly. The controlled Rust case shows that even a correct output can be blocked by the harness. The frontend case shows that a confident low-randomness output can still be wrong.

The M2 Max's 32 GB shared memory is a genuine constraint. Model weights, context/KV cache, images, macOS, and other applications share the same physical budget. MoE active parameters affect computation per token; they do not mean only those active weights need residency. Large storage does not remove the RAM requirement.

The context estimator uses heuristics and a fixed per-image estimate, then calibrates against observed prompt usage. It is not an exact tokenizer or a precise multimodal memory model. A comfortable configuration must account for actual resident memory and pressure, not just nominal parameter count or a context number in the model description.

The fresh live work here used Qwen m2max-fast:latest. It did not rerun the full experimental matrix on Gemma. Historical Gemma failures and current provider tests are discussed as separate evidence. This report does not claim a cross-model success rate or recommend buying hardware to solve the confirmed harness defects.

15. Latency and the experience of waiting

The user's complaint is not just speed. Waiting while an agent makes no visible useful progress is worse than waiting for a large correct result. In the historical repair, large old_text and new_text payloads consumed generation time, repeated reads and failed edits added round trips, and the final answer still did not describe the final file accurately.

Thinking effort increases opportunity for reasoning; it does not guarantee better decisions. Higher effort can amplify a bad plan or spend more time reproducing large source text. A useful small edit plus a targeted check is often cheaper and more reliable than a long reasoning cycle followed by a whole-file replacement.

The current scheduling policy distinguishes Off, Nominal, and High, uses observed throughput where available, and has transport/task limits. A configuration nuance remains: a model timeout numerically equal to the default 120 seconds is treated as adaptable. A direct scheduling probe produced 180/240/360-second allowances for that value. Therefore explicitly choosing 120 cannot be distinguished from leaving the default at 120 in that path. Other explicit values were preserved in the probe.

This is a configuration semantics issue, not evidence that every timeout is ignored. It should be documented or represented explicitly if the UI promises a strict timeout. Model preparation, command execution, transport deadlines, iteration limits, and whole-task duration are also distinct. Per-inference limits do not alone establish a short total task duration.

Progress communication should describe observable activity: inspecting a file, preparing a patch, waiting for approval, running a check, or needing a decision. It should not fabricate internal reasoning. An animation makes waiting visible but does not repair a stalled loop. More frequent generic 'considering your request' text does not tell the user what remains blocked.

The desired efficient experience follows from smaller work units and meaningful state: act, check, summarize what changed, and ask only for a decision that changes the result. This can be done with the existing execution events and request_user_input path; it does not need a second model generating commentary.

16. Native UI, lifecycle, builds, and restart policy

The native application is substantial and has useful tested components: chat input, Markdown display, diff presentation, approval cards, zoom handling, model state, Job Apps, and lifecycle controls. Fresh Swift release compilation succeeded. A native assertion harness compiled current production views/helpers and executed 91 test methods successfully.

The standard xcrun swift test command did not run successfully because the installed Command Line Tools environment could not provide XCTest. This is an environment/test-runner blocker, not a failed app assertion. The fallback harness is useful but is not the official XCTest engine. The report keeps those results separate rather than saying 'all Swift tests passed.'

The unmodified native acceptance runner initially failed for an important reason: it only approved the old run_command tool name. The application now requested exec_command correctly, and the runner denied it. Lorca honored the denial and stopped. That is test fixture drift combined with correct denial behavior, not a model command-schema failure.

An audit-only copy of the runner was then adjusted to recognize both names, use the current model and nominal effort, and keep bounded diagnostic timeouts. It passed the full native flow: update README, repair src/lib.rs, run Cargo tests with exit zero, and answer a follow-up accurately. Production source was not changed to make that pass.

Existing restart-policy tests all passed. The lifecycle/rebuild work addresses genuine risks from old processes, replaced binaries, reconnecting clients, and application shutdown. It does not repair wrong model output or verification semantics. A fresh backend can execute the same flawed authority or completion rule perfectly consistently.

The audit compiled fresh Rust and Swift release targets but did not replace/restart the user's running production application or claim an installed bundle had been repackaged. Builds establish compilation, not complete UI polish across every screen and zoom level. Previously reported click/padding issues should not be declared impossible on the basis of a compile or isolated helper tests.

17. Job Apps: strengths and boundaries of the evidence

Job Apps is a specialized workflow with research, application tracking, recruiter contacts, outreach, follow-up state, resume selection/tailoring, and Google synchronization. Its independent registry and data flow help keep domain-specific work separate from the general chat agent. That separation is valuable and should not be undone to fix general coding reliability.

The fresh Python suite ran 111 tests successfully. These tests provide evidence for internal behavior, persistence/sync logic, conflict handling, and controlled integration boundaries. They are not proof that real Google authorization, every Gmail thread, every Sheet conflict, every recruiter source, and continuous research all worked live during this audit.

No real emails were sent and no live Google Sheet was updated for this report. This matters because '111 tests passed' can otherwise sound like external integrations were comprehensively exercised. Credentials, network state, API enablement, account permissions, and simultaneous-device conflicts remain separate operational variables.

The user-facing Job Apps design has developed around useful concrete actions: oldest pending job, add a recruiter/email when research cannot find one, review outreach, track replies and follow-up dates, select a resume, and manually complete application forms. Those are more bounded than an arbitrary 'do anything on my computer' request, which is one reason specialized flows can be more dependable.

There is also documentation and expectation drift. Some text describes manual Gmail review while backend capabilities include explicitly approved sending. A report should not assume either is universally exposed identically in every installed build. The operational contract should say what this version can send, what requires approval, what attachment it uses, and which state syncs.

Nothing in the confirmed general-agent defects requires redesigning Job Apps, changing its sidebar, or rewriting its business logic. Shared provider/model/lifecycle changes can still affect both areas, so regression coverage matters. The appropriate action is to preserve the specialized workflow while validating its external behavior separately when that is the task at hand.

18. Modularity review: good structure, remaining coupling

The five tool drawers are physically present, the registry is central, and tool handlers are independently named. Shared filesystem, command/session, argument, and HTTP logic avoids obvious duplication. This is close to the intended simple architecture and should be kept.

There are still coupling points. The agent loop and verification code contain tool-specific branches for evidence, restoration, progress, and required outcomes. Adding a new tool can be easy at registration time but still require orchestration changes if it introduces a new kind of verifiable result. That is a maintainability limitation, not a reason for a plugin framework.

A second extension hazard is the distinction between declared authority metadata and prepared-action approval. The registry describes authority, but actual execution approval is controlled by prepared tool behavior. A future handler that declares approval-required metadata but prepares an unapproved action incorrectly could violate the intended extension contract. This audit did not demonstrate a current shipping permission bypass. The concern is preventing future registration/handler inconsistency with focused contract tests.

The approximate inspected footprint was 2,687 lines across eight agentloop files, 2,817 across 26 tool files, 1,527 across six core files, 5,116 across 47 Swift source files, and 4,219 across 26 Job Apps Python files including tests. These counts describe breadth, not an objective measure of overengineering.

The main complexity is behavioral: several representations of task intent, question state, completion requirements, provider budgets, persisted history, and UI statuses must agree. More layers would probably make that harder. A small number of shared authoritative records and explicit transitions would make it easier.

The working tree also contains significant uncommitted and untracked implementation. A clean checkout at HEAD does not necessarily reproduce the audited tool chest or latest recovery behavior. Release provenance is therefore an operational blocker: testing one tree and pulling another can make the user believe a model behaves inconsistently when the binaries or sources differ. No commit was made in this audit; the report records the exact distinction.

19. Why development has felt like endless repair

The first problem was overclaiming readiness. A green test suite, a valid tool call, or a successful patch proves something useful, but not the entire user workflow. Earlier completion statements were too broad. The correct claim would have been 'this infrastructure checkpoint works under these cases; end-to-end reliability remains unproven.' That distinction would have prevented surprise and frustration.

The second problem was symptom-led repair. An exec_command argument failure prompted schema guidance work; a malformed Ollama response prompted output recovery; a bad patch prompted exact-match discussion; a long pause prompted thinking UI. Each can be legitimate, but without a stable set of real acceptance cases the work keeps moving to the latest symptom instead of proving the core loop.

The third problem was brittle interpretation in the harness. Conservative enforcement is good, but global phrase matching can invent restrictions, and keyword-based outcome detection can miss obvious requests. That creates failures even when the model has the right plan. These defects can be mistaken for model intelligence or context problems because the visible result is simply an incomplete task.

The fourth problem was missing behavioral oracles. Most mechanical checks ask whether a patch applied, a process ended, or state persisted. Frontend behavior needs a different check. No amount of file-write correctness establishes that a slider changes colors. The audit's Safari test demonstrates why a legal JavaScript program can still be broken.

The fifth problem was scope expansion and integration churn. UI polish, Google sync, resume workflows, zoom, lifecycle, model switching, and the general tool chest all became active development areas. These features were requested and useful; the issue is that they expanded the regression surface before a dependable daily coding baseline was established.

Finally, live tests drifted. An ignored test expected the old Incomplete question status. The native runner approved an obsolete tool name. A long fixture referenced old paths. Passing ordinary tests hid those problems because the end-to-end tests were optional or stale. The remedy is a small maintained acceptance gate, not thousands of additional unit tests that repeat implementation assumptions.

20. Ranked bottlenecks and blockers

Priority 1: active task semantics. Scoped prohibitions must not become global bans; a follow-up must retain the current requested outcome; explicit cancellation/new work must replace a pending objective. These are confirmed by direct probes and fresh live experiments. They block ordinary careful use and cannot be solved by changing models.

Priority 1: truthful completion. 'Applied and read back' must not be conflated with 'behavior verified.' Running a failing test suite can satisfy a reporting task. An edit request must not have an empty requirement list because of phrasing. Final summaries must use final observations rather than intermediate ones.

Priority 1: patch strategy and current-content awareness. Exact matching alone allowed a large destructive prefix removal. The agent should operate on small relevant regions against current contents, preserve required structure, and re-read/check after changes. A user-approved mechanical patch is not a semantic certificate.

Priority 2: clarification and collaboration. Native questions work, but reasonable plain-text questions loop. Generic failure questions and real waiting questions behave differently. The user wants to choose meaningful directions, not repeatedly repair internal agent state.

Priority 2: practical tool coverage. Cargo-only execution and no browser verification leave normal frontend tasks without the needed checks. The right solution is a narrowly scoped set of real capabilities with existing authority, not exposing arbitrary power without controls or building an entire browser product before any improvement is possible.

Priority 2: release/test reproducibility. Dirty working-tree features, stale acceptance fixtures, and unavailable standard XCTest make confidence hard to transfer between machines and builds. A known source/build identity and maintained native/live gate are necessary operational discipline.

Priority 3: adaptive resource/configuration clarity. Context estimates, optional thinking support, timeout semantics, image costs, and memory pressure need clear limits. These matter, but they are not the demonstrated root cause of the broken frontend repair.

Not the immediate blocker: lack of another reasoning engine, absence of subagents, insufficient UI animation, or a need to rewrite Job Apps. The strongest evidence points to smaller repairs around the existing architecture. Model quality remains relevant, but it must be measured under a correct harness rather than blamed for every failure.

21. Smallest practical repair sequence

This is a recommendation, not work implemented by this audit. First freeze the current code and capture the failing cases from this report as independent acceptance tests. Preserve the working successes alongside them. A change should not be accepted merely because one new test passes while a native flow or a user cancellation regresses.

Checkpoint A should repair task state and scope together: represent the active requested outcomes and allowed targets concisely; distinguish an answer to a pending question from cancellation or a new assignment; and distinguish run-and-report from repair-and-verify. Keep ordinary conversation ordinary. Do not build a general natural-language policy engine.

Checkpoint B should make collaboration predictable. Reuse request_user_input and existing UI. Recognize or steer ordinary clarification into one clear waiting state. Ask when a preference changes the result, recommend an option, remember the answer, and continue. Do not require the user to approve every harmless investigation or explain every routine tool error.

Checkpoint C should make edits bounded and verification honest. Read relevant current content, apply small changes, and record a content/version relationship where practical. Detect suspicious structural loss or an edit outside the intended region as a reason to inspect, not as a universal ban on large changes. Report applied, checked, and unverified aspects separately without introducing a complex scoring framework.

Checkpoint D should add only the execution/checking capabilities needed for the chosen daily workflow. For frontend work that includes actual browser/CSS behavior checks or a clear human visual checkpoint. Keep session lifecycle, cancellation, bounded output, workspace scope, and approval. No unrelated UI work is necessary.

Checkpoint E should validate the supported model set with the same tasks, ordinary language variants, and independently checked outcomes. Keep the shared contract; negotiate optional thinking and vision capabilities. Update fixtures when public tool names or statuses change. Publish only a reproducible source/build state that passes the agreed gate.

Do not try to solve every future computer task in one release. A dependable read/edit/check/clarify/resume workflow would be a meaningful finished milestone. The present architecture can support that without another planner, another agent, or a broad rewrite.

22. A simple daily workflow and a realistic definition of done

For a frontend request, Lorca should first inspect the relevant file and identify the requested visible behaviors. If 'rainbow slider' could mean angle rotation or hue selection, it should ask a concise question with a recommendation. If the user already specifies hue checkpoints, it should act without repeating that question.

It should then make one small relevant edit, such as replacing the slider's color calculation while preserving controls. It should check syntax and the resulting CSS values, and use a browser or an explicit user visual check for appearance. It can then fix dark-mode styling as a separate bounded change. The checkpoint records what remains, not a transcript of internal thoughts.

If it cannot launch a browser with the available tools, it should say: 'The code change is applied and the syntax check passed; I have not visually verified the page. Please move the slider and tell me whether the colors match.' That is honest collaboration. It should not label the visual behavior as tested, and it should not ask the user to restate the entire task.

For a Rust task, requested-file scope and allowed checks should be clear. 'Do not edit other files' should preserve the target edit. A successful fix should include an actual relevant test result. A request to report existing failures should stop after reporting them rather than trying to force exit zero.

A finished milestone should pass: empty-file creation; repository inspection; exact-path read recovery; scoped Rust repair in both positive and negative wording; run-and-report failing tests; a question followed by an answer; a pending question followed by cancellation/new work; a small frontend behavior fixture; malformed tool generation recovery; and a follow-up explanation based on final evidence.

It should also fail safely: deny prohibited paths, honor declined approval, never execute partial JSON, bound repeated bad calls, and preserve useful work on cancellation. Perfect model output is not the standard. Accurate state, bounded recovery, and an honest result are. That is the practical route to 'simple, modular, powerful, and reliable.'

23. Fresh automated tests and build results

All results below are from this audit, not copied from an earlier completion claim. Commands ran against /Users/deve/Downloads/lorca unless a subdirectory is noted. Full command records and logs are retained in the evidence folder.

- cargo test: exit 0, 37.09 seconds; 175 passed, zero failed, five ignored. Ignored live tests are not included as passes.
- cargo clippy --all-targets -- -D warnings: exit 0, 7.69 seconds.
- cargo fmt --check: exit 0, 0.24 seconds.
- Python unittest discovery in jobapps/tests: exit 0, 111 tests passed, 1.98 seconds. Controlled/mock external integration coverage, not real Gmail/Sheets validation.
- Python unittest discovery in macos/Tests/Restart: exit 0, six tests passed, 0.13 seconds.
- cargo build --release: exit 0, 20.60 seconds.
- xcrun swift build -c release, macos package: exit 0, 15.48 seconds.
- Native Swift assertion harness: compiled successfully, then 91 test methods passed, 15.73 seconds execution. This uses an audit-only assertion shim over current test methods, not official XCTest execution.
- xcrun swift test: exit 1, 3.95 seconds; XCTest unavailable in the installed Command Line Tools environment. This remains an unfulfilled standard test-runner check.

Opt-in model tests were run separately. The existing Qwen live suite had one pass and one failure. Empty README creation passed. The unspecified-file question assertion failed because it still expected Incomplete when the actual native-question status was WaitingForInput. That failure is fixture drift, not evidence that the question tool failed.

The separate thinking smoke test passed for Off, Nominal, and High by generating valid read_file calls. It does not execute a full coding task and must not be counted as three coding successes.

The original native acceptance failed when its obsolete run_command-only approval policy denied exec_command. An audit-only adapted runner passed all three stages after recognizing the current tool name. These results are reported separately so a diagnostic adaptation does not hide the original maintenance defect.

No single combined pass percentage is meaningful across unit tests, mocked integration tests, model generations, native stages, and direct probes. They exercise different things and have different failure interpretations.

24. Fresh live experiments and known limits

Nine scenario records cover eleven user submissions through the real Core/backend with Qwen m2max-fast:latest in disposable projects. Nominal effort was used with explicit bounded model/command limits. These are diagnostic experiments, not a statistically representative model benchmark.

- Unspecified file request, first run: Incomplete in 15.05 seconds. Sensible ordinary-text clarification was repeated instead of cleanly recognized. No file written.
- Repeated unspecified-file setup: Incomplete in 10.44 seconds. Same recognition problem. The planned follow-up stage was not reached in these two cases.
- Broad frontend request A: Completed in 28.75 seconds. Structure and JS syntax passed; seven fixed colors rotated rather than a hue sweep.
- Broad frontend request B: Completed in 32.27 seconds. Same 708-byte output and behavior, demonstrating repeatability of the chosen interpretation.
- Rust repair with 'Do not edit other source files': WaitingForInput after 94.70 seconds. Correct patch blocked; independent tests still failed with exit 101.
- Explicit native question then answer: WaitingForInput in 5.77 seconds, then Completed in 7.03 seconds; exact empty CHOSEN.md created.
- Explicit native question then cancellation/new task: WaitingForInput in 5.36 seconds, then Incomplete in 16.45 seconds; obsolete edit objective retained.
- Rust repair with positive target scope: Completed in 16.35 seconds; independent cargo test exited 0.
- Precise frontend hue request: Completed in 26.47 seconds; JS syntax passed but generated CSS failed actual Safari validation at all four sampled positions.

A direct mechanics probe separately exercised requirement inference, failed-test evidence, authority preparation, terminal allowlists, and timeout scheduling. Its destructive-prefix patch was only prepared for approval, not executed. It demonstrates a mechanical contract gap, not an unauthorized write.

Ten saved conversations contained 46 persisted agent-result records: 33 Completed, eight Incomplete, five Failed; 34 Qwen and 12 Gemma records. These include greetings, earlier software versions, and simple tasks. They are not a 72% success rate. The production incident itself shows why the Completed label cannot serve as a quality oracle.

The audit did not establish long-chat reliability, full Gemma parity, real Google sync/email correctness, every UI interaction at every zoom, overnight research stability, or absence of all security bugs. The report is exhaustive about the inspected evidence and major architectural boundaries, not a claim of exhaustive state-space testing.

25. Evidence index, reproducibility, and final judgment

All audit artifacts are under /Users/deve/Documents/Codex/2026-10-24/pu/outputs/exhaustive-audit/. Key records are checks.json, native-checks.json, live-existing.json, core-live-matrix.json, mechanics-probe.json, frontend-behavior.json, browser-css-validation.json, incident-repairs.json, saved-chat-inventory.json, git-history.txt, git-status.txt, and source fingerprint files. Logs retain the individual command output and native/model traces. They contain no newly sent messages or production edits.

Relevant source entry points, relative to /Users/deve/Downloads/lorca: src/agentloop/authority.rs for scope checks; verification.rs for requirements and question recognition; execution.rs and recovery.rs for continuation/failure behavior; objective.rs and runtime.rs for task state/checkpoints; src/core/agent_chats.rs for pending-question linkage; src/core/generation.rs and the provider/model code for generation policy; src/context.rs for token estimation; src/tools/registry.rs for discovery; src/tools/workspace.rs and files/apply_patch.rs for filesystem edits; terminal/command.rs and terminal/sessions.rs for execution; web/http.rs for public fetch constraints.

Native areas include macos/Sources/LorcaMac/ChatStore.swift, AgentStore.swift, AgentClient.swift, Shutdown.swift, AppZoom.swift, and the JobApps-prefixed views/store/bridge. Native acceptance is under macos/Acceptance; Rust live tests include tests/agent_live.rs and tests/agent_thinking_live.rs. The report's audit-only adaptations live outside the production repository.

For reproduction, use the same working tree and installed model, not just HEAD. Run the ordinary test/build commands listed earlier. Execute live tests serially to avoid model contention. Use disposable data directories and workspaces. The external core_live_matrix.py and additional_live.py scripts record prompts, statuses, files, and independent checks; they are diagnostic tools, not additions to the shipped product.

The before/after fingerprint comparison covered 166 implementation/test files and found no differences. It does not imply build caches or temporary files were unchanged: compilation deliberately produced those. The actual damaged playground file was read/copied for investigation, not repaired.

Final judgment: this is not a failed concept or a need to start over. Lorca has working execution infrastructure and demonstrable successful workflows. It remains unfinished as a reliable daily general agent because task interpretation, collaboration state, edit strategy, available checks, and completion truth do not consistently agree. The next milestone should make those few contracts dependable and prove them with real user-style cases. Keep the architecture, keep the tools modular, keep Job Apps intact, and stop equating mechanical completion with a finished user result.